Программирование в лингвистике

Лекция 16

Pandas: Производительность

DataFrame.apply Vs NumPy

```
import numpy as np
import pandas as pd

df = pd.DataFrame({
    "Age": np.random.randint(1, 80, 1000000)
})
#df['Age'].apply(classify_age)
np.where(df['Age'] < 18, 'Minor', 'Adult')
```

Многие функции Pandas можно заменить конструкциями на основе np.array и получить значительный прирост в производительности.

✓ Метод DataFrame.apply() в Pandas работает значительно медленнее, чем векторизированные операции в NumPy, так как apply() фактически запускает пошаговый цикл на чистом Python внутри себя для каждой строки или элемента.

✓ Идут накладные расходы из-за работы глобального интерпретатора Python.

✓ Также он создает промежуточные объекты Series на каждой итерации, которые требуют больше времени на обработку и дополнительно расходуют память.

Конвертация: метод to_numpy

```
import pandas as pd
# Импорт numpy здесь не нужен!

df = pd.DataFrame({"Цены": [10, 20, 15.5]})
arr = df['Цены'].to_numpy(dtype='float32')
arr[0] = 100
```

```
arr_view[0] = 100
   ~~~~~~~~^^^
ValueError: assignment destination is read-only
```

Мы создали вьюшку, а не оригинал массива!

✓ **Series.to_numpy()** позволяет сделать из объектов pandas массивы numpy.

✓ Поскольку pandas основан на numpy, это делается очень быстро.

✓ По сути Series — это массив, на который сверху «надели» обертку в виде индексов и дополнительных методов. Метод .to_numpy() просто снимает эту обертку и дает прямой доступ к чистым данным.

✓ У этого метода всего два основных параметра, но они фундаментально влияют на то, как расходуются ресурсы.

Конвертация: метод to_numpy

```
import pandas as pd

df = pd.DataFrame({"Цены": [10, 20, 15.5]})
arr = df['Цены'].to_numpy(copy=False)
arr.flags.writeable = True
arr[0] = 100

print(df.iloc[0])
```

```
Цены    100.0
Name: 0, dtype: float64
```

```
import pandas as pd

df = pd.DataFrame({"Цены": [10, 20, 15.5]})
arr = df['Цены'].to_numpy(copy=True)
arr[0] = 100

print(df.iloc[0]))
```

```
Цены    10.0
Name: 0, dtype: float64
```

Ловушка разных типов данных

✓ DataFrame.to_numpy(), примененный к массиву с разными типами данных в столбцах, будет приводить данные к одному типу (upcasting).
✓ Это может повлечь за собой нежелательные результаты, как в примере ниже.
✓ Поэтому чаще используется DataFrame[name].to_numpy() – или со срезами.

```
import pandas as pd

bad_df = pd.DataFrame({"Возраст": [25, 30, 45], "Имя": ["Анна", "Иван", "Олег"]})

arr = bad_df.to_numpy() # Применится upcasting!
print(arr)
```

```
[['25' 'Анна'] ['30' 'Иван'] ['45' 'Олег']]
```

Category для экономии памяти

✓ Использование типа данных **category** в pandas позволяет значительно сократить расход оперативной памяти и ускорить операции (например, groupby) за счет внутреннего представления строк в виде массива целых чисел (кодов) и словаря уникальных значений.
✓ Экономия огромна, если количество уникальных значений столбца во много раз меньше числа строк (например, столбец со статусами заказов, полом, регионами или городами на миллионы записей).

Как это работает:
• Тип object (стандартный для строк) - каждая строка хранится как отдельный объект Python в памяти, что расходует много лишнего объема.
• Тип category - Pandas создает две сущности: массив индексов (меньшего размера, например, int8 или int16) и массив уникальных значений (словарь/categories).

Category для экономии памяти

```
import numpy as np
import pandas as pd

cities_pool = ['Москва', 'Санкт-Петербург', 'Новосибирск', 'Екатеринбург', 'Казань']
random_cities = np.random.choice(cities_pool, size=1000000)
df = pd.DataFrame({'city': random_cities})
print(df.memory_usage(deep=True) / 1024**2) # ~ 82 MB
df['city'] = df['city'].astype('category')
print(df.memory_usage(deep=True) / 1024**2) # ~ 1 MB
```

Параметр deep=True: без этого параметра **df.memory_usage()** оценивает только объем указателей, а не реальный вес строк в памяти Python.

Chunksize для толстых данных

✓ Бывает, что данные слишком велики, чтобы помещаться в оперативную память, но все еще слишком малы, чтобы называться Big data. Такие данные будем называть толстыми (англ. large but not big). Термин отсюда.

✓ В этом случае нам поможет чтение большими кусками с указанием **chunksize**.

✓Чтобы дополнительно сократить количество используемой памяти, отфильтруйте только необходимые столбцы. Можете сразу применить агрегации.

```
import pandas as pd

chunk_size = 10000
with pd.read_csv('large_file.csv', usecols=use_cols, chunksize=chunk_size) as reader:
    for chunk in reader:
        # какие-то агрегации
```

cProfile и поиск узких мест

✓ Человеческая интуиция часто плохо оценивает скорость работы кода. Мы можем часами оптимизировать функцию, которая занимает всего 1% общего времени работы программы.
✓ Нам нужен инструмент, который точно измерит время работы каждого фрагмента программы. Он называется профайлер (profiler), а стандартный профайлер в Python — модуль **cProfile**.

Как использовать cProfile:

- Можно запустить его из командной строки: python -m cProfile my_script.py
- Или вызвать в самом коде:

```
import cProfile
cProfile.run('my_function()')  # Выдаст отчет в терминал
```

cProfile и поиск узких мест

Столбец	Что значит	На что смотреть
ncalls	Сколько раз была вызвана функция.	Если число огромное — возможно, функция в лишнем цикле.
tottime	Чистое время работы самой функции (без учета вызовов других функций внутри нее).	Помогает найти «виновника» тормозов внутри кода.
percall	tottime / ncalls	Среднее время на один вызов.
cumtime	Кумулятивное время. Считает функцию и всё, что она вызвала внутри себя.	У main() оно всегда будет самым большим.
filename:lineno (function)	Имя файла, номер строки и название функции.	Помогает понять, куда смотреть в коде.

Спасибо за внимание!